Multiple Linear Regression

This model, also known as **linear regression with multiple features**, is an extension of the model from Week 1. It allows us to use more than one input feature to make more accurate predictions.

Motivation 🤔

Instead of predicting a house's price based only on its size, we can use additional information to improve our model.

- **Single Feature (Week 1):** x = size
- **Multiple Features (Week 2):**
  - x_1 = size
  - x_2 = number of bedrooms
  - x_3 = number of floors
  - x_4 = age of home

New Notation ✍️

With multiple features, we need to update our notation to handle them.

- x_j: Denotes the **j-th feature** in our feature list (e.g., x_2 is the number of bedrooms).
- **n**: The **total number of features** (in our example, n=4).
- $\vec{x}^{(i)}$: Represents the **i-th training example**. This is now a **vector**, or a list of numbers, containing all the feature values for that example.
  - For example, $\vec{x}^{(2)} = [1416, 3, 2, 40]$.
- $x_j^{(i)}$: The value of the **j-th feature** in the **i-th training example**.
  - For example, $x_3^{(2)} = 2$ (the number of floors in the second house).

The Updated Model Equation

The model is extended from a simple line to an equation that incorporates all n features.

- **Model with n features:**
  $f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$

To write this more compactly, we use vector notation and the **dot product**.

1. **Define Parameter and Feature Vectors:**

- The parameters `w` are now a vector: $\vec{w} = [w_1, w_2, \ldots, w_n]$
- The features `x` for a single example are a vector: $\vec{x} = [x_1, x_2, \ldots, x_n]$
- `b` remains a single number (a scalar).

2. **Use the Dot Product:** The dot product of $\vec{w}$ and $\vec{x}$ is defined as:
   $\vec{w} \cdot \vec{x} = w_1x_1 + w_2x_2 + \cdots + w_nx_n$
3. **The Final, Compact Model:**
   $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

Vectorization

Vectorization is the practice of implementing machine learning algorithms using numerical linear algebra libraries (like **NumPy** in Python) instead of explicit loops. This makes your code **shorter** and allows it to **run much more efficiently**.

Why is Vectorization Faster? 🚀

- Vectorized operations use **parallel hardware** in your computer's CPU (Central Processing Unit) or GPU (Graphics Processing Unit).
- Libraries like NumPy are highly optimized to perform calculations on entire arrays at once, which is significantly faster than a standard Python `for` loop that processes one element at a time.

Example: Implementing the Linear Regression Model

Let's compare three ways to calculate the model's prediction: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$.

1. Non-Vectorized (Manual) Approach

This involves manually multiplying and adding each element.

```
f = w[0] * x[0] + w[1] * x[1] + w[2] * x[2] + b
```

- **Problem:** This is tedious to write and impractical if you have a large number of features (`n`).

---

2. Non-Vectorized (For Loop) Approach

This is an improvement but is still not efficient.

```
f = 0
for j in range(0, n):
  f = f + w[j] * x[j]
f = f + b
```

- **Problem:** Standard Python `for` loops are slow because they process calculations sequentially, one after another.

---

3. Vectorized Approach ✅

This uses the **NumPy** library to perform a **dot product**, which is the most efficient method.

```
import numpy as np

f = np.dot(w, x) + b
```

- This single line of code is equivalent to the loops above but runs much faster. It leverages NumPy's optimized, parallel computations.

Key Takeaway

Vectorization provides two main benefits:

1. **Simpler Code:** Your code becomes much shorter and easier to read.
2. **Faster Execution:** Your code runs significantly faster, especially with large datasets.

How Vectorization Works: Parallel vs. Sequential Processing

Vectorization feels like a "magic trick" because it fundamentally changes how your computer performs calculations, leveraging modern hardware for massive speed gains.

Non-Vectorized (Sequential) Processing 🚶

- A standard `for` loop executes **sequentially**.
- This means it processes one instruction at a time, one after another. If you need to perform 16 calculations, the computer does the first one, then the second, then the third, and so on, taking 16 distinct time steps.

Vectorized (Parallel) Processing 🚀

- Vectorized operations, like those in NumPy, use **parallel processing**.
- The computer's hardware (CPU or GPU) is designed to perform the same operation on all elements of an array **at the same time, in a single step**.
- For example, to multiply two vectors with 16 elements, the hardware can perform all 16 multiplications simultaneously. It then uses specialized hardware to sum the results very efficiently.
- This parallel execution is why vectorization is dramatically faster, especially for large datasets. It can be the difference between an algorithm finishing in minutes versus taking many hours.

Vectorizing the Gradient Descent Update

This principle can be directly applied to the gradient descent update rule for the weight parameters.

- **Goal:** Update each parameter w_j according to the rule: $w_j := w_j - \alpha \cdot d_j$ (where d_j is the derivative for that parameter).

Non-Vectorized (For Loop) Approach

You would loop through each parameter and update it individually.

```
# Slower, sequential update
for j in range(n):
  w[j] = w[j] - alpha * d[j]
```

Vectorized Approach ✅

You treat `w` and `d` as vectors (or NumPy arrays) and perform the update in a single line of code.

```
# Faster, parallel update
w = w - alpha * d
```

Behind the scenes, the computer performs all the subtractions and multiplications for every element of the `w` vector at the same time.

Gradient Descent for Multiple Linear Regression

This section combines the concepts of multiple linear regression and vectorization to create an efficient implementation of the gradient descent algorithm.

Review: Vector Notation ✍️

To simplify our equations, we group our parameters and features into **vectors**.

- **Parameters:**
  - Instead of separate parameters $w_1, w_2, \ldots, w_n$, we define a single parameter **vector** $\vec{w} = [w_1, w_2, \ldots, w_n]$.
  - `b` remains a single number (a scalar).
- **Model:** The model can be written compactly using the **dot product**: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
- **Cost Function:** The cost function is now a function of the vector $\vec{w}$ and the scalar `b`: $J(\vec{w}, b)$

The Gradient Descent Algorithm

The algorithm updates all `n` of the `w` parameters and the `b` parameter simultaneously on each iteration.

- **Update Rule for w_j (for j=1...n):** $w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$
- **Update Rule for b:** $b := b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$

Derivative Formulas for Multiple Regression

The specific derivative terms for the multiple linear regression cost function are very similar to the univariate case.

- **Derivative with respect to w_j:** $\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$
- **Derivative with respect to b:** $\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$

The main difference from the single-feature model is that we now have a separate update for each feature's weight (w_1, w_2, etc.), and the derivative for w_j specifically uses the corresponding feature value, x_j.

Alternative Method: The Normal Equation (Side Note) 📝

- For linear regression *only*, there is a non-iterative method for finding the optimal w and b called the **Normal Equation**.
- It involves solving for the parameters directly using advanced linear algebra, without needing to choose a learning rate α or run multiple iterations.
- **Disadvantages:**
  - It **does not generalize** to other algorithms like logistic regression or neural networks.
  - It can be very **slow** if the number of features (n) is large.
- **Conclusion:** Gradient descent is a more general and often more practical method for most machine learning applications.